



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Herramienta de gestión de
calendario EPS**



Presentado por Rebeca Cuesta Estépar
en Universidad de Burgos — 27 de mayo
de 2026

Tutor: Álvaro Arnaiz González, Ana Serrano
Mamolar



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. nombre Álar Arnáiz González y Dña. nombre Ana Serrano Mamolar, profesores del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos.

Expone:

Que la alumna Dña. Rebeca Cuesta Estépar, con DNI 71705464C, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Herramienta de Gestión de Calendario EPS.

Y que dicho trabajo ha sido realizado por la alumna bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 27 de mayo de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

Dña. Ana Serrano Mamolar

D. Álar Arnaiz González

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	iv
Índice de tablas	v
1. Introducción	1
2. Objetivos del proyecto	3
2.1. Objetivos funcionales	3
2.2. Objetivos técnicos	5
3. Conceptos teóricos	7
4. Técnicas y herramientas	11
4.1. Lenguajes y entornos de programación	11
4.2. Base de datos y despliegue	11
4.3. Frameworks	12
4.4. Bibliotecas	15
4.5. Otras herramientas y técnicas	15
5. Aspectos relevantes del desarrollo del proyecto	17
6. Trabajos relacionados	21
7. Conclusiones y Líneas de trabajo futuras	23
Bibliografía	25

Índice de figuras

Índice de tablas

3.1. Herramientas y tecnologías utilizadas en cada parte del proyecto	10
---	----

1. Introducción

Este Trabajo de Fin de Grado (TFG) pretende resolver la problemática de centralizar toda la gestión de aulas y de horarios del campus Río Vena de la Escuela Politécnica de la Universidad de Burgos. Se ha decidido realizar este trabajo por los beneficios que presentaría para el cliente, en este caso, la Escuela de la Politécnica de la Universidad de Burgos, el disponer de una aplicación web en la que poder gestionar los horarios del campus, las reservas, el calendario de exámenes de convocatorias y la ocupación de las aulas, todo en una misma aplicación.

Esta aplicación, de la que ahora mismo la universidad no dispone de algo parecido, es muy necesaria ya que facilita y automatiza en gran medida, tanto el trabajo de la realización de los horarios de cada campus, como la gestión de reservas, todo ello vinculado a la ocupación de aulas. Y es que no es nada práctico, por ejemplo, tener toda esta información almacenada en hojas de cálculo, ya que cuando se mueve una reserva o una asignatura de día u hora, se tiene que ir a la ocupación del aula y cambiar también manualmente, por el contrario, esta aplicación lo hace de manera automática, disminuyendo el trabajo y por consecuencia minimizar el error humano. Además de que cuando alguien realiza una solicitud de una reserva o directamente se crea una, el sistema nos muestra qué aulas se encuentran disponibles para la fecha y hora seleccionadas, evitando así tener que ir mirando aula por aula la disponibilidad para el plazo especificado.

Además, esta aplicación también nos permite la exportación de los horarios de los distintos grados, los exámenes de convocatoria por grado, la ocupación de las aulas semanal por las asignaturas (reservas periódicas) y las reservas puntuales por aula.

2. Objetivos del proyecto

El objetivo general de este proyecto es diseñar e implementar una página web que permita centralizar y optimizar la gestión de horarios, reservas de aulas y planificación académica del campus, mejorando la organización y gestión interna y facilitando la consulta de información tanto a los gestores como al profesorado.

Este proyecto, consiste en diseñar e implementar una aplicación web destinada a las personas relacionadas con la gestión de reservas en las facultades de la Universidad de Burgos, en concreto se ha limitado en esta primera versión al campus Vena. En el acceso a la aplicación se distinguirán diferentes roles: administrador, gestor editor, gestor no editor y el personal docente investigador (PDI). Según el rol del usuario que inicie sesión, se le permitirá acceder a unas funcionalidades u otras, según los permisos que disponga.

2.1. Objetivos funcionales

1. **Gestión de reservas puntuales:** Los usuarios autenticados, sin importar su rol, podrán solicitar una reserva indicando la fecha y la hora, el motivo y las características que necesita que tenga el aula. Estas solicitudes serán gestionadas por los gestores, quienes podrán editar los datos de la reserva, y modificar el aula escogido para la reserva. Además, estas reservas dispondrán de un estado (pendiente, aprobada y rechazada) y únicamente el gestor editor podrá crear, modificar o eliminar reservas ya existentes. Cada modificación realizada en ellas, deberá reflejarse automáticamente en la ocupación del aula correspondiente si se viera afectado.

2. **Consulta de aulas y ocupación:** El sistema permitirá consultar la ocupación de un aula en un rango de fechas concreto, mostrando tanto las reservas puntuales como las periódicas (ambas o filtrar), y generar una vista semanal basada en las reservas periódicas con el objetivo de crear un informe que poder colocar en la puerta del aula con los detalles de las asignaturas, grupos y horarios.
3. **Consulta de reservas:** La aplicación incluye un apartado de consulta de reservas, que permite visualizar y filtrar todas las reservas puntuales registradas en el sistema, para facilitar su seguimiento y control.
4. **Gestión de usuarios y roles:** Esto será responsabilidad del administrador, que podrá crear, modificar y eliminar usuarios, así como asignarles el rol correspondiente para poder garantizar un control adecuado de los permisos y de las acciones permitidas dentro del sistema.
ESPERA DEL RESULTADO PARA EL LOGIN
5. **Gestión de exámenes y generación de convocatorias:** En este módulo, a partir de datos de convocatorias anteriores, se implementará un mecanismo de rotación que permita generar nuevas convocatorias manteniendo las fechas y horas establecidas, pero rotando las asignaturas que se examinan para que no tengan siempre el mismo orden. El sistema permitirá generar informes de exámenes por grado y curso, así como realizar alguna modificación excepcional cuando sea necesario.
6. **Gestión de horarios y reservas periódicas:** Este es uno de los objetivos más importante del proyecto. El sistema permitirá consultar horarios por grado y curso, así como importar horarios de cursos anteriores como base sobre la que modificar. Permitiendo realizar modificaciones de forma manual sobre el horario, que comprobará si se aplican y cumplen una serie de restricciones que se encontrarán previamente definidas, como la imposibilidad de solapamiento de asignaturas para un mismo grupo, o que se encuentre dentro del horario de apertura de la politécnica. Cuando una modificación vulnere alguna restricción, el sistema mostrará una advertencia, permitiendo al usuario confirmar la acción bajo su responsabilidad y avisando de la violación.

2.2. Objetivos técnicos

1. **Modelo de bases de datos relacional:** se ha diseñado para permitir el almacenamiento de forma estructurada de la información relativa a aulas, grados, asignaturas, reservas, horarios, exámenes y usuarios.
2. **Arquitectura web:** su implementación se ha basado en la separación entre el *frontend* y el *backend*, favoreciendo de esta forma la modularidad y mantenibilidad
3. **Sistema de permisos:** se ha implementado un sistema de control de acceso basado en roles que limita las acciones disponibles en la aplicación según el tipo de usuario.
4. **Cálculo de disponibilidad:** que garantiza la actualización de manera automática de la ocupación de las aulas ante cualquier cambio que se realice en las reservas o los horarios.
5. **Generación de informes:** permitiendo extraerlos en formato pdf para su posterior publicación.
6. **Validación de restricciones:** al realizar una acción se comprueba que no viole ninguna de las restricciones definida.
7. **Calidad:** se realizan validaciones y manejo de errores.

3. Conceptos teóricos

En esta sección de la memoria, se pretenden recoger los conceptos teóricos más relevantes para la comprensión del proyecto.

Aplicaciones web y arquitecturas cliente-servidor

Una aplicación web es un sistema software al que se puede acceder a través de un navegador, más concretamente a través de una URL, con lo que requiere tener conexión a Internet. Además está sustentada sobre una arquitectura cliente-servidor.

Esta arquitectura o modelo tiene funciona de la siguiente manera.

- El cliente se encarga de la interacción con el usuario y la representación de la interfaz, es decir, es el que hace las peticiones de servicios al servidor y recibe las respuestas de estas.
- El servidor es le que centraliza la lógica de negocio, el acceso a los datos, y recibe y procesa las peticiones del cliente etnregándoles una respuesta.

La comunicación entre el cliente y el servidor se realiza por medio de protocolos de red. En las aplicaciones web, el más común es *HTTP* (*Hyper Text Transfer Protocol*), un protocolo de comunicación que define cómo se intercambian las peticiones y respuestas entre cliente y servidor. Gracias a este mecanismo, el cliente puede solicitar o enviar recursos o información al servidor, mientras que este puede responder con datos o el resultado de una operación concreta. [3]

Esta separación de responsabilidades facilita el mantenimiento, la escalabilidad y la evolución de forma independiente de las distintas partes que conforman el sistema. En este proyecto se utiliza un framework distinto para el *front* (cliente) que para el *back* (servidor).

API REST

Una API (*Application Programming Interface*) es un conjunto de reglas, protocolos y definiciones que permite la comunicación entre diferentes sistemas software. En el contexto de las aplicaciones web, una API permite que el *frontend* y el *backend* intercambien información de una forma estructurada, sin la necesidad de que ambos softwares estén implementados con la misma tecnologías.

En particular, una API REST (*Representational State Transfer*) es un estilo de arquitectura ampliamente utilizado en el desarrollo de servicios web. Su objetivo principal es organizar la comunicación entre cliente y servidor mediante recursos identificados por URLs y operaciones estándar del protocolo HTTP. De esta forma permite que los recursos del sistema como, en el caso de este proyecto, un aula o una reserva, pueda ser modificado o consultado a través de una dirección concreta y utilizando unos métodos bien definidos.

Entre los métodos más comunes de HTTP en una API REST se encuentran:

- *GET*: utilizado para solicitar o recuperar información.
- *POST*: envía datos al servidor para crear un nuevo recurso.
- *PUT* o *PATCH*: para modificaciones de recursos ya existentes.
- *DELETE*: utilizado para la eliminación de recursos existentes.

Con esta organización las operaciones del sistema quedan estructuradas. [2]

Patrón arquitectónico MVT

Django se apoya en el patrón arquitectónico MVT (*Model-View-Template*), una variante del modelo de separación de responsabilidades que es utilizada de forma frecuente en el desarrollo de aplicaciones web. Este tipo de patrón permite estructurar el sistema separando la gestión de los datos, el procesamiento de las peticiones y la presentación de información, favoreciendo así una organización más clara y mantenible del código.

- El componente Modelo (*Model*) representa la estructura de datos y la interacción con la base de datos. En Django, los modelos son la fuente principal de información sobre los datos almacenados y permiten definir tanto sus atributos como ciertos comportamientos asociados. Además se integran con el ORM (*Object-Relational Mapping*) del *framework*, lo que facilita la forma de trabajar con la base de datos ya que en vez de hacerlo a través de consultas SQL de manera exclusiva, se puede realizar mediante objetos del propio lenguaje.
- La Vista (*View*) es el elemento encargado de recibir una petición HTTP, procesarla y devolver una respuesta. En ella se encuentra la lógica necesaria para decidir qué información debe obtenerse, validarse o transformarse antes de ser enviada al cliente. De esta forma, la vista actúa como un intermediario entre los datos definidos en los modelos y la respuesta final que recibe el usuario o la aplicación cliente.
- La Plantilla (*Template*) se encarga de la presentación y visualización de la información. Su función es definir cómo presentar los datos al usuario, separando así la lógica de presentación del resto de responsabilidades del sistema. Aunque en el caso de este proyecto la interfaz de usuario se implementa con React y, por tanto, la representación gráfica del lado del cliente no recae sobre las plantillas de Django. Este patrón sigue siendo útil a un nivel conceptual, debido a que el *backend* mantiene la separación entre accesos a datos, lógica de negocio y exposición de servicios.

El uso de este patrón arquitectónico ayuda a reducir el acoplamiento entre componentes, facilitando el mantenimiento del sistema y mejorando su escalabilidad. Asimismo, permite que las distintas partes de la aplicación puedan evolucionar de una forma más independiente, algo que es bastante relevante en proyectos en los que el *backend* se desarrolla de manera desacoplada al *frontend*. [1] [4]

Herramientas	App	AngularJS	API REST	BD	Memoria
HTML5		X			
CSS3		X			
BOOTSTRAP		X			
JavaScript		X			
AngularJS		X			
Bower		X			
PHP			X		
Karma + Jasmine		X			
Slim framework			X		
Idiorm			X		
Composer			X		
JSON		X	X		
PhpStorm		X	X		
MySQL				X	
PhpMyAdmin				X	
Git + BitBucket		X	X	X	X
MikTeX					X
TeXMaker					X
Astah					X
Balsamiq Mockups		X			
VersionOne		X	X	X	X

Tabla 3.1: Herramientas y tecnologías utilizadas en cada parte del proyecto

4. Técnicas y herramientas

En este apartado se van a explicar las decisiones tomadas a lo largo del desarrollo del proyecto, en cuanto las herramientas y técnicas elegidas y utilizadas en este. También se va a exponer en el caso de que se hayan valorado más opciones la motivación de decidirse por la herramienta finalmente elegida frente a otras opciones.

4.1. Lenguajes y entornos de programación

Para la realización de este proyecto, se ha decidido utilizar *python* como lenguaje de programación, por su gran popularidad en la actualidad, la importancia que está tomando en el mundo de la programación y su gran demanda en el mundo laboral. Con lo que puede ser interesante para profundizar más en su funcionamiento y mejorar y conocer más a fondo este lenguaje de programación, ampliando los conocimientos del mismo que se adquirieron en las asignaturas de Sistemas Inteligentes y Algoritmia.

Además, el entorno de desarrollo en el que se va a realizar todo el proyecto, tanto la parte del *backend* como la del *frontend*, va a ser *Visual Studio Code*, por ser principalmente un editor de código gratuito, por su alto rendimiento y por tener soporte para múltiples lenguajes de programación. Permitiendo así tanto el desarrollo web como el *backend*, además de que permite una amplia personalización del entorno mediante diferentes extensiones.

4.2. Base de datos y despliegue

En esta sección se exponen las herramientas elegidas para soportar el funcionamiento de la aplicación, tanto a nivel de almacenamiento de datos

como de despliegue de la aplicación. Asimismo, se justifican las elecciones realizadas en función de la compatibilidad con el framework empleado y de las necesidades de mantenimiento del sistema.

Sistema gestor de base de datos: MySQL

La aplicación requiere de una base de datos para almacenar y consultar toda la información relacionada horarios, reservas, y aulas. Para llevarlo a cabo se ha elegido MySQL como Sistema Gestor de Base de Datos (SGBD), por su amplio uso y por tener compatibilidad directa con Django mediante *subbackend*. Además, Django documenta soporte oficial para MySQL a partir de versiones modernas.

Con lo que he elegido MySQL ya que a nivel de mantenimiento permite un control independiente de la versión del sistema gestor de bases de datos, evitando que me limite la compatibilidad con versiones del *framework* del *backend* elegido.

Servidor y entorno de ejecución: Apache

Se han valorado dos alternativas: Apache y XAMPP. XAMPP es un paquete que tiene todo en uno, es decir, te proporciona en el mismo programa el sistema gestor de base de datos y el servidor web, incluyendo Apache y MariaDB. Sin embargo, XAMPP distribuye versiones concretas de sus componentes, lo que limitaba la obsolescencia del proyecto ya que las nuevas versiones de Django requieren versiones del sistema gestor de base de datos más actualizadas para poder tener compatibilidad. Por lo que, haber escogido XAMPP como opción, hubiese tenido que instalar versiones de Django anteriores, lo que aceleraría el proceso de desfase de la aplicación.

Por la razón anterior, y la independencia que da descargar por separado el sistema gestor de base de datos y el servidor web, se ha decidido instalar Apache.

4.3. Frameworks

En este apartado se recogen las decisiones tecnológicas relacionadas con los *frameworks* empleados en el proyecto, tanto para el *backend* como para el *frontend*. Su selección no solo responde a criterios técnicos, sino también a la necesidad de contar con una solución mantenible, flexible y alineada con el tipo de aplicación web que se pretende desarrollar.

Backend: Django

Una vez seleccionado Python como lenguaje principal del proyecto, se decidió utilizar Django como *framework* de desarrollo para el *backend*. La elección fue tomada principalmente por la necesidad de una arquitectura modular, mantenible y escalable, se tuvo en cuenta que el sistema debía organizarse en componentes independientes (por ejemplo, gestión de horarios, gestión de aulas, reservas, calendario, etc.) con responsabilidades bien delimitadas, y por ser un proyecto basado en el desarrollo de una aplicación web.

Django favorece este enfoque mediante su organización basada en aplicaciones, permitiendo dividir el proyecto en módulos reutilizables e independientes, con el fin de dividir la lógica del proyecto, facilitando el crecimiento del mismo y reduciendo el acoplamiento entre distintas funcionalidades.

Además, Django se apoya en el patrón arquitectónico MVT (Modelo-Vista-Plantilla), que guía la separación de responsabilidades:

- **Modelo (Model):** define la estructura de datos y la interacción con la base de datos mediante el ORM (*Object-Relational Mapping*).
- **Vista (View):** contiene la lógica de negocio y gestiona las peticiones/respuestas, es decir, procesa las solicitudes que entran por parte de los usuarios y
- **Plantilla (Template):** se encarga de la presentación de los datos al usuario; en este proyecto, esta separación sigue siendo útil aunque el *frontend* se implemente con React, ya que el *backend* se centra en exponer servicios y lógica.

Backend: Django REST Framework

Dado que se necesita que el *frontend* consuma datos del *backend*, concluimos que iba a ser necesaria la utilización de una API, mediante *endpoints*. Por ello decidió instalarse Django REST Framework (DRF).

Esta elección se debe a que DRF proporciona componentes esenciales para este tipo de arquitectura. Entre los más relevantes destacan:

- **Serializadores:** permiten transformar instancias de modelos y consultas en formatos intercambiables (por ejemplo JSON) y, además, validar los datos de entrada antes de persistirlos.

- **Vistas y ViewSets:** facilitan la implementación de endpoints reutilizables y coherentes, reduciendo código repetido y estandarizando operaciones habituales (listar, crear, modificar y eliminar recursos).
- **Autenticación y permisos:** permiten controlar el acceso a la API, aplicando políticas de seguridad según el tipo de usuario o rol.

Frontend: React + Vite

Para llevar a cabo el desarrollo del *frontend*, se tuvieron en cuenta dos opciones: *React* y *Angular*.

Aunque verdaderamente *React* se define como una librería para construir interfaces de usuarios, basada en componentes, permitiendo reutilizar piezas y separar de manera clara las funcionalidades de la interfaz. Además, nos permite una mayor flexibilidad de cara a la estructura del proyecto, permitiendo seleccionar únicamente las dependencias necesarias. Por otro lado, tenemos *Angular* un *framework* que es más cerrado, con decisiones y herramientas integradas, teniendo una mayor carga inicial y la adaptación a la forma de trabajo es menos flexible.

Con lo que finalmente elegí utilizar *React*, porque me pareció importante la característica de la flexibilidad. Una vez seleccionado *React*, investigué en profundidad y encontré dos formas de iniciarlo en el proyecto: *Create React App* y *Vite + React*. Tras buscar información relativa a ambas opciones, finalmente elegía la opción que contiene Vite.

Y es que Vite es una alternativa más moderna, de hecho el propio equipo de *React* no recomienda actualmente su utilización debido a limitaciones, y recomienda utilizar herramientas integradas, como es el caso de la escogida.

Esta herramienta destaca por el hecho de que proporciona un entorno de desarrollo ligero y rápido al contar con un servidor de desarrollo optimizado y capacidades como la recarga en caliente (*HMR - Hot Module Replacement*) que es instantáneo sin importar lo grande que sea la app, lo que mejora el flujo de trabajo durante la implementación. Además, *Vite* utiliza *Rollup* que es muy rápido en eliminar el código muerto que no usas, resultando en una aplicación final más ligera mejorando las métricas de rendimiento web.

CSS: Tailwind

Para la estética del frontend valoré dos alternativas: *Tailwind CSS* y *Bootstrap*.

Tras analizar la documentación sobre ambos *frameworks*: *Bootstrap* permite crear interfaces de forma más rápida debido a que cuenta con un catálogo de componentes predefinidos (botones, formularios,...), pero debido a esta característica en muchas ocasiones las aplicaciones que hacen uso de este *framework* utilizando su configuración por defecto tienen una estética muy parecida. Se puede personalizar pero requiere estar sobrescribiendo. Por otro lado, tenemos que Tailwind CSS es *utility-first*, es decir, se basa en clases atómicas que se combinan para construir diseños a medida. Este enfoque permite una mayor flexibilidad y no depende de estilos preestablecidos, además encaja con el desarrollo por componentes del *frontend*. Por todos estos motivos, fue por los que elegí este *framework* para los estilos del proyecto.

4.4. Bibliotecas

Biblioteca de Calendarios: Fullcalendar

En cuanto a la biblioteca de calendarios que va a ser utilizada para el *frontend*, se han valorado dos opciones: *Fullcalendar* y *React Big Calendar*.

Tras estudiar los recursos que ofrece cada una de estas bibliotecas se ha determinado que la elegida iba a ser *Fullcalendar*. Aunque *React Big Calendar* es una biblioteca *opensource*, y por lo tanto, totalmente gratuita se ha optado por elegir la otra, ya que con las funcionalidades que ofrece de manera gratuita es suficiente para la realización de este proyecto y además, ofrece mucha mayor documentación y viene implementada ya la funcionalidad del *drag and drop*.

4.5. Otras herramientas y técnicas

Scrum

Github

Zenhub

TeXstudio

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

En esta sección de la memoria se busca resumir y resaltar aspectos teóricos y prácticos sobre la implementación y decisiones relevantes del proyecto.

Analizador léxico

Durante el desarrollo del proyecto, se decidió realizar un analizador léxico para la carga de los horarios de los grados en el sistema, los cuales se encuentran ahora mismo en una hoja de cálculo. Esta decisión fue tomada

debido a que para poder modificar la franja horario en la que se imparte una clase tiene que estar cargada en base de datos, y habría que cargar de manera manual las reservas en el sistema. Lo que no es viable ya que con la extracción de los datos de la hoja de cálculo, el número de clases o reservas extraídas supera las 1000 reservas periódicas.

Para comenzar se analizó en profundidad la estructura que presentaban las clases teóricas y prácticas de la hoja de cálculo que se utiliza actualmente para la gestión de los horarios. Tras el análisis se supuso que las clases teóricas podían ser encontradas con tres estructuras diferentes, y las clases prácticas con otras tres estructuras, con una estructura de ellas común entre los dos tipos de clases. También se observó que las clases teóricas tenían un color y las prácticas otro a nivel general incluyendo las clases en inglés del grado de Organización Industrial. También se analizaron los datos que era importante extraer para poder realizar las cargas en base de datos de las reservas periódicas, entre los que se encuentran, la asignatura, su código (identificador de la asignatura en base de datos), el grupo, el aula, el día de la semana y la franja horaria en la que se imparte la clase.

Durante el desarrollo de este *parser*, se detectó que la hoja de cálculo inicial presentaba una estructura poco homogénea. Aunque a nivel general, sí que seguía una estructura, se encontraron datos introducidos sin mecanismos de validación consistentes, y en ocasiones sin un criterio uniforme. Por consecuencia, se realizó una fase de depuración, normalización y unificación de la información, que supuso una cantidad de tiempo considerable a lo largo del desarrollo para que pudiera ser integrada de manera correcta en el sistema. Para poder comprender el tiempo invertido cabe destacar que los valores cargados en los horarios se introducían o modificaban a través de validadores de datos, en este caso listas o desplegables. Con lo que cada vez que se modificaba el valor de uno de estos datos en la hoja correspondiente, había que ir a las hojas de los horarios respectivas en las que se encontraba el dato y volver a seleccionarlo para que se actualizara, esto para cada una de las celdas que contenía ese dato.

Entre los problemas de unificación encontrados están los siguientes:

1. Inconsistencia en el formato de colores. Aunque visualmente había colores que podían parecer similares, la tonalidad o código era distinto para representar una misma categoría. El grupo teórico de inglés tenía el mismo color designado que el grupo teórico para el segundo grupo de teoría.

- Solución aplicada: Se decidió qué color y tonalidad iba a tener cada categoría encontrada, y
2. Introducción manual de datos. Se encontraron datos que en vez de haber sido añadidos mediante el validador se habían introducido a mano en la celda, en vez de ser añadidos en la lista de celdas de la que el validador extraía los datos.
 - Solución aplicada: Se analizó y detectó que datos habían sido introducidos de manera manual, se añadió su valor a la lista del validador, y se eligió mediante este.
 3. Inconsistencia en la nomenclatura de los laboratorios. Para referirse a los laboratorios, se utilizaban diferentes abreviaturas (Lab., Lab, Lb., L.). La última de estas abreviaturas provocaba que alguna asignatura la identificase como aula por acabar con «l». Por ejemplo, APL. BASES DE DATOS (Aplicación de base de datos).
 - Solución aplicada: Se unificó la abreviatura a Lab. o Lab para todos los laboratorios. Y posteriormente, se tuvo que actualizar en cada celda en la que se encontraba uno de los laboratorios modificados.
 4. Inconsistencia en la separación de aulas combinadas. Las combinaciones de aulas eran separadas por diferentes caracteres especiales («,», «/», «-»)
 - Solución aplicada: Se unificó el separador de las aulas a «/» en el validador, y se actualizó en cada celda que aparecía la combinación de todos los horarios para que se aplicaran los cambios.
 5. Inconsistencia en las abreviaturas de las asignaturas. Se encontró que las abreviaturas de las asignaturas no eran homogéneas. La misma asignatura podía encontrarse en el mismo horario con tilde y sin ella, o directamente la abreviatura fuese totalmente diferente. Por ejemplo, se podía encontrar: AMP. CALC., AMP. CÁLCULO o AMP. CALCULO
 - Solución aplicada: Unificar la abreviatura de las asignaturas en la hoja de Asignaturas, e ir actualizando en el horario del grado y semestre correspondiente.
 6. Valores con fuente blanca. Se encontraron celdas en las que el fondo era blanco y la fuente también, es decir, estos valores existían pero no se veían.

- Solución aplicada: Se detectaron que valores se imprimían que no entraban dentro de las estructuras predefinidas con colores, y se puso el color de la fuente a negro y se eliminaron los valores.
7. Filas ocultas. Se encontraron filas ocultas que estaban dando error porque no tenían los nombre unificados, y tras tiempo analizando bien la situación resultó que no se veían y estaban ocultas.
- Solución aplicada: Se expandieron todas las filas en las que se producía salto y no eran contiguas, y se eliminaron.
8. Clases sin aula asignada. Algunas clases no tienen definida el aula por no conocerse si finalmente van a ser impartidas o no, corresponden a asignaturas optativas. Pero para poder realizar la reserva, es obligatorio introducir un aula.
- Solución aplicada: Se implementó en las distintas estructuras del analizador léxico que si no se encontraba un valor de tipo Aula en la celda correspondiente se le iba a asignar el valor Aula 0. Y en base de datos se añadió un Aula 0 que no existe, solamente es provisional para poder realizar la reserva.

Tras terminar la fase preparación y unificación de la hoja de cálculo, y finalizar de forma correcta la implementación del *parser*, para poder cargar las reservas en base de datos, hubo que cargar previamente todas las asignaturas y aulas con sus respectivos campos en base de datos.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] MDN contributors. Introducción a django. https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Django/Introduction, 15/04/2026. [Internet; visitado 27-mayo-2026].
- [2] Oscar Morales Cuellar. Métodos http: Todo lo que debes de conocer sobre los métodos de petición http. <https://tecsify.com/blog/metodos-http/>, 27/02/2024. [Internet; visitado 12-mayo-2026].
- [3] Fernán García de Zúñiga. Todo sobre la arquitectura cliente-servidor. <https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor>, 17/07/2025. [Internet; visitado 12-mayo-2026].
- [4] Geeks for Geeks. Django project mvt structure. <https://www.geeksforgeeks.org/python/django-project-mvt-structure/>, 02/05/2026. [Internet; visitado 27-mayo-2026].